



Universidad de las Fuerzas
Armadas ESPE

Departamento de Ciencias de la
Computación



Pruebas Unitarias para el Sistema TransControl

Docente: Ing. Jenny Alexandra Ruiz Robalino

Asignatura: Análisis y Diseño de Software

NRC: 30724

Integrantes: Jaguaco Jonathan
Ñacato Alexander
Obando Erick
Saraguro Leidy



Sangolquí - Ecuador
8 de julio de 2026

Índice

1. Objetivos de las Pruebas	2
1.1. Objetivo General	2
1.2. Objetivos Específicos	2
2. Contexto de Pruebas	2
2.1. Alcance de las Pruebas	2
2.2. Alcance de las Pruebas	2
2.3. Enfoque de Testing	3
3. Herramienta	3
3.1. Framework de Testing	3
3.2. Herramientas Adicionales	3
3.3. Entorno de Pruebas	3
3.4. Estructura de Archivos	4
4. Implementación por Requisito	4
4.1. RF001: Inicio de Sesión	4
4.2. RF002: Gestionar Transportista	6
4.3. RF003: Gestionar Viajes	8
4.4. RF004: Monitorear Viajes	10
4.5. RF005: Crear Transportista	12
4.6. RF006: Ingresar Documentacion	13
4.7. RF007: Verificar Documentación	15
4.8. RF008: Recuperar Contraseña	16
4.9. RF009: Editar Transportista	18
4.10. RF010: Eliminar Transportista	19
4.11. RF011: Crear Viajes	21
4.12. RF012: Consultar Viajes	22
4.13. RF013: Creacion de la Base de Datos	23
4.14. Resultados	25
5. Resumen Final	26
6. Referencias	27

1. Objetivos de las Pruebas

1.1. Objetivo General

Validar mediante pruebas unitarias la lógica de negocios y componentes de software del sistema TransControl asociadas a las tareas de:

- La lógica CRUD en la gestión de cuentas de transportistas.
- Los algoritmos de cifrado, creación y recuperación de credenciales.
- Las reglas de validación de negocio para la aceptación de documentación.
- Los métodos de cambio de estado y cálculo de asignación de envíos/rutas.

1.2. Objetivos Específicos

- Probar cada uno de los 13 requisitos funcionales del sistema.
- Descargar e instalar las herramientas de Python necesarias para las pruebas unitarias.
- Crear un entorno virtual de Python para ejecutar las pruebas.
- Crear un directorio test en el cual se agregaran los archivos .py respectivo de cada prueba unitaria.
- Documentar cada prueba para su posterior descripción y análisis.
- Colocar las referencias de los documentos que ayudaron a realizar las pruebas unitarias

2. Contexto de Pruebas

2.1. Alcance de las Pruebas

Las pruebas unitarias realizadas en el sistema TransControl cubren los siguientes aspectos:

2.2. Alcance de las Pruebas

Las pruebas unitarias automatizadas implementadas en el sistema TransControl se delimitan estrictamente a la lógica de la aplicación del lado del servidor (*backend*), aislando cualquier dependencia externa como servidores de bases de datos o servicios de mensajería de red. Los aspectos evaluados se estructuran a nivel de los siguientes directorios y módulos del proyecto:

- **Modelos (models/models.py):**
Verificación de la integridad estructural de los esquemas, la existencia de los campos críticos de control, las restricciones de integridad relacional de la información.
- **Controladores y Rutas (routers/)**
Los requisitos funcionales del 14 a 16 no se contemplan dentro de este documento por ende los requisitos funcionales del 1 al 13 contemplan únicamente autorización, autenticación y flujo del negocio.
- **Esquemas y DTOs (schemas/)**
Se comprueba que las restricciones de entrada no rechacen cargas útiles malformadas antes de interactuar con la lógica interna es decir el flujo de información del sistema.

- **Componentes de Adaptación y Servicios (patterns/)**

Validación de los servicios y patrones de diseño integrados como lógica dentro del sistema siendo tanto verificación de los adaptadores de infraestructura para geolocalización (`CityToGPSAdapter`), las estrategias dinámicas de rastreo de rutas (`UbicacionStrategy`) y las funciones extras de utilidad y solución dentro del sistema.

2.3. Enfoque de Testing

- **Metodología de Desarrollo:** TADD (Test - Assisted Development / Desarrollo Asistido por Pruebas)
- **Tipos de Pruebas:** Unitarias
- **Cobertura Objetivo:** 90 % (12 de 13 Requisitos Funcionales)
- **Cobertura Real:** 100 % (13 de 13 Requisitos Funcionales)
- **Metodología de Diseño de Pruebas:** Patrón AAA (Arrange, Act, Assert)

3. Herramienta

3.1. Framework de Testing

Pytest: Framework basado en Python para realizar pruebas unitarias

3.2. Herramientas Adicionales

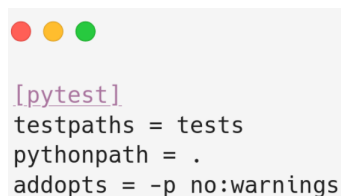
Pytest-cov: Librería para medir el porcentaje del código fuente abarcado para cada prueba.

Configuración Adicional: Instanciación de librerías dentro de `requirements.txt` para la ejecución de las pruebas



```
# Testing
pytest
pytest-cov
```

Creación del archivo `pytest.ini` para ejecutar las pruebas unitarias en Python

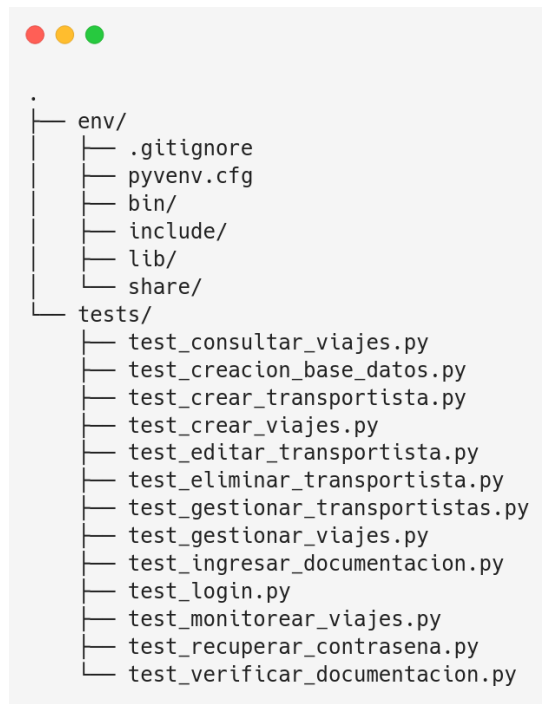


```
[pytest]
testpaths = tests
pythonpath = .
addopts = -p no:warnings
```

3.3. Entorno de Pruebas

- **Versión de Python:** 3.14.5
- **Env:** Entorno virtual aislado para Python
- **Sistema Operativo Host:** Fedora Linux

3.4. Estructura de Archivos



4. Implementación por Requisito

4.1. RF001: Inicio de Sesión

Descripción: Autenticación de usuarios según su rol asignado, garantizando el ingreso correcto al sistema y la restricción estricta de las vistas a sus funciones exclusivas.

- **Archivo Test:** test_login.py
- **Total de Pruebas:** 8 pruebas
- **Estado:** 100 % PASSED

Implementación

```

1  # tests/test_login.py
2  from unittest.mock import Mock, patch
3  import pytest
4  from fastapi import HTTPException
5  from routers.auth import login
6
7  class TestLogin:
8      """Pruebas unitarias para RF-1: Iniciar Sesión (login) con control de roles."""
9
10     # Usamos parametrización para generar 5 pruebas dinámicas (una por cada rol requerido)
11     @pytest.mark.parametrize("rol_usuario", ["CLIENTE", "COORDINADOR", "TRANSPORTISTA", "SECRETARIA", "GERENTE"])
12     def test_login_exitoso_roles(self, rol_usuario):
13         body = Mock(username="usuario@correo.com", password="clave123")
14         request = Mock()
15         request.client.host = "127.0.0.1"
16
17         user_mock = Mock(
  
```

```

18         id=1,
19         rol=rol_usuario,
20         nombres="Jonathan",
21         activo=True,
22         hashed_password="hash_falso",
23     )
24
25     db = Mock()
26     db.query.return_value.filter.return_value.first.return_value = user_mock
27
28     with patch("routers.auth.verify_password", return_value=True), \
29         patch("routers.auth.create_access_token", return_value="token_falso"), \
30         patch("routers.auth.registrar_auditoria"):
31
32         resultado = login(body, request, db)
33
34         assert resultado.access_token == "token_falso"
35         assert resultado.rol == rol_usuario
36         assert resultado.nombres == "Jonathan"
37         assert resultado.id == 1
38
39     def test_login_usuario_no_existe(self):
40         body = Mock(username="noexiste@correo.com", password="clave123")
41         request = Mock()
42         request.client.host = "127.0.0.1"
43
44         db = Mock()
45         db.query.return_value.filter.return_value.first.return_value = None
46
47         with pytest.raises(HTTPException) as exc_info:
48             login(body, request, db)
49
50         assert exc_info.value.status_code == 401
51         assert exc_info.value.detail == "Correo y/o contraseña incorrectos"
52
53     def test_login_password_incorrecta(self):
54         body = Mock(username="usuario@correo.com", password="claveMala")
55         request = Mock()
56         request.client.host = "127.0.0.1"
57
58         user_mock = Mock(id=1, rol="GERENTE", activo=True, hashed_password="hash_falso")
59
60         db = Mock()
61         db.query.return_value.filter.return_value.first.return_value = user_mock
62
63         with patch("routers.auth.verify_password", return_value=False):
64             with pytest.raises(HTTPException) as exc_info:
65                 login(body, request, db)
66
67             assert exc_info.value.status_code == 401
68             assert exc_info.value.detail == "Correo y/o contraseña incorrectos"
69
70     def test_login_cuenta_inactiva(self):
71         body = Mock(username="usuario@correo.com", password="clave123")
72         request = Mock()
73         request.client.host = "127.0.0.1"
74
75         user_mock = Mock(id=1, rol="TRANSPORTISTA", activo=False, hashed_password="hash_falso")
76
77         db = Mock()
78         db.query.return_value.filter.return_value.first.return_value = user_mock
79
80         with patch("routers.auth.verify_password", return_value=True):

```

```

81         with pytest.raises(HTTPException) as exc_info:
82             login(body, request, db)
83
84         assert exc_info.value.status_code == 403
85         assert "inactiva" in exc_info.value.detail.lower()

```

Salida en Consola

```

tests/test_login.py::TestLogin::test_login_exitoso_rol[CLIENTE] PASSED
[ 12%]
tests/test_login.py::TestLogin::test_login_exitoso_rol[COORDINADOR] PASSED
[ 25%]
tests/test_login.py::TestLogin::test_login_exitoso_rol[TRANSPORTISTA] PASSED
[ 37%]
tests/test_login.py::TestLogin::test_login_exitoso_rol[SECRETARIA] PASSED
[ 50%]
tests/test_login.py::TestLogin::test_login_exitoso_rol[GERENTE] PASSED
[ 62%]
tests/test_login.py::TestLogin::test_login_usuario_no_existe PASSED
[ 75%]
tests/test_login.py::TestLogin::test_login_password_incorrecta PASSED
[ 87%]
tests/test_login.py::TestLogin::test_login_cuenta_inactiva PASSED
[100%]

===== 8 passed in 1.39s =====

```

4.2. RF002: Gestionar Transportista

Descripción: Visualización detallada de transportistas y su estado por parte del Coordinador.

- **Archivo Test:** test_gestionar_transportista.py
- **Total de Pruebas:** 8 pruebas
- **Estado:** 100 % PASSED

Implementación

```

1  # tests/test_gestionar_transportistas.py
2  from unittest.mock import Mock
3  import pytest
4  from fastapi import HTTPException
5  from routers.transportistas import listar_transportistas, obtener_transportista
6
7  # Helpers compactos para la generación de dobles de prueba (Mocks)
8  def _base_mock(t_id=1, nombres="Carlos", activo=True, documentos=None):
9      usuario = Mock(cedula="0102", nombres=nombres, correo="c@c.com", telefono="099", direccion="
10         UIO", activo=activo)
11      t = Mock(id=t_id, usuario_id=10, usuario=usuario, placa_vehiculo="ABC", tipo_vehiculo="Camión",
12         capacidad_ton=5.0, documentos=documentos or [])
13      return t
14
15  def _doc_mock(doc_id=1, tipo="CEDULA", estado="APROBADO"):
16      return Mock(id=doc_id, tipo=tipo, nombre_archivo="f.pdf", estado=estado, fecha_vencimiento=
17         None, observacion=None, subido_en="2026", revisado_en=None, contenido_pdf=b"")

```

```

15
16 class TestGestionarTransportistas:
17     """Pruebas unitarias agrupadas para la gestión y consulta de transportistas."""
18
19     # -- PRUEBAS DE LISTAR TRANSPORTISTAS (GET /api/transportistas/) --
20
21     def test_listar_transportistas_exitoso_con_datos(self):
22         t1 = _base_mock(1, "Carlos Pérez", documentos=[_doc_mock(1, "CEDULA", "APROBADO"),
23         _doc_mock(2, "LIC", "PENDIENTE")])
24         t2 = _base_mock(2, "María López")
25         db = Mock()
26         db.query.return_value.join.return_value.filter.return_value.order_by.return_value.all.return_value = [t1, t2]
27
28         res = listar_transportistas(solo_activos=True, db=db, current_user=Mock(rol="COORDINADOR"))
29
30         assert len(res) == 2
31         assert res[0]["nombres"] == "Carlos Pérez"
32         assert res[0]["estado_documentacion"] == "PENDIENTE"
33         assert res[1]["estado_documentacion"] == "SIN_DOCS"
34
35     def test_listar_transportistas_lista_vacia(self):
36         db = Mock()
37         db.query.return_value.join.return_value.filter.return_value.order_by.return_value.all.return_value = []
38
39         res = listar_transportistas(solo_activos=True, db=db, current_user=Mock(rol="COORDINADOR"))
40
41         assert isinstance(res, list) and len(res) == 0
42
43     def test_listar_transportistas_incluye_inactivos(self):
44         db = Mock()
45         db.query.return_value.join.return_value.order_by.return_value.all.return_value = [
46         _base_mock(1, activo=True), _base_mock(2, "Pedro", activo=False)]
47
48         res = listar_transportistas(solo_activos=False, db=db, current_user=Mock(rol="COORDINADOR"))
49
50         assert res[0]["activo"] is True and res[1]["activo"] is False
51
52     def test_listar_transportistas_detalle_completo(self):
53         t = _base_mock(5, "Ana Torres", documentos=[_doc_mock(10, "SOAT", "APROBADO")])
54         db = Mock()
55         db.query.return_value.join.return_value.filter.return_value.order_by.return_value.all.return_value = [t]
56
57         res = listar_transportistas(solo_activos=True, db=db, current_user=Mock(rol="COORDINADOR"))[0]
58
59         for campo in ["id", "cedula", "nombres", "placa_vehiculo", "tipo_vehiculo", "capacidad_ton", "estado_documentacion"]:
60             assert campo in res
61
62     # -- PRUEBAS DE OBTENER UN TRANSPORTISTA (GET /api/transportistas/{id}) --
63
64     def test_obtener_transportista_exitoso(self):
65         db = Mock()
66         db.query.return_value.filter.return_value.first.return_value = _base_mock(7, "Luis Ramírez", documentos=[_doc_mock(3, "SOAT", "APROBADO")])
67
68         res = obtener_transportista(transportista_id=7, db=db, current_user=Mock(rol="COORDINADOR"))
69
70         assert res["id"] == 7 and res["nombres"] == "Luis Ramírez" and res["estado_documentacion"] == "APROBADO"
71
72     def test_obtener_transportista_no_encontrado(self):
73         db = Mock()

```



```

65         db.query.return_value.filter.return_value.first.return_value = None
66
67         with pytest.raises(HTTPException) as exc:
68             obtener_transportista(transportista_id=999, db=db, current_user=Mock(rol="
COORDINADOR"))
69         assert exc.value.status_code == 404
70
71     def test_obtener_transportista_con_documentos_rechazados(self):
72         t = _base_mock(3, documentos=[_doc_mock(1, "CED", "APROBADO"), _doc_mock(2, "LIC", "
RECHAZADO")])
73         db = Mock()
74         db.query.return_value.filter.return_value.first.return_value = t
75
76         res = obtener_transportista(transportista_id=3, db=db, current_user=Mock(rol="
COORDINADOR"))
77         assert res["estado_documentacion"] == "RECHAZADO"
78
79     def test_obtener_transportista_sin_documentos(self):
80         db = Mock()
81         db.query.return_value.filter.return_value.first.return_value = _base_mock(4, documentos
=[])
82
83         res = obtener_transportista(transportista_id=4, db=db, current_user=Mock(rol="
COORDINADOR"))
84         assert res["estado_documentacion"] == "SIN_DOCS"

```

Salida en Consola

```

tests/
test_gestionar_transportistas.py::TestGestionarTransportistas::test_listar_transportistas_exitoso_con_d
atos PASSED
tests/
test_gestionar_transportistas.py::TestGestionarTransportistas::test_listar_transportistas_lista_vacia
PASSED
tests/
test_gestionar_transportistas.py::TestGestionarTransportistas::test_listar_transportistas_incluye_inact
ivos PASSED
tests/
test_gestionar_transportistas.py::TestGestionarTransportistas::test_listar_transportistas_detalle_compl
eto PASSED
tests/test_gestionar_transportistas.py::TestGestionarTransportistas::test_obtener_transportista_exitoso
PASSED
tests/
test_gestionar_transportistas.py::TestGestionarTransportistas::test_obtener_transportista_no_encontrado
PASSED
tests/
test_gestionar_transportistas.py::TestGestionarTransportistas::test_obtener_transportista_con_documento
s_rechazados PASSED
tests/
test_gestionar_transportistas.py::TestGestionarTransportistas::test_obtener_transportista_sin_documento
s PASSED

===== 8 passed in 0.12s =====

```

4.3. RF003: Gestionar Viajes

Descripción: Visualizar los viajes y su información por cada viaje

- **Archivo Test:** test_gestionar_viajes.py
- **Total de Pruebas:** 10 pruebas
- **Estado:** 100 % PASSED

Implementación

```

1  # tests/test_gestionar_viajes.py
2  from unittest.mock import Mock, patch
3  import pytest
4  from fastapi import HTTPException
5  from routers.monitoreo import obtener_ubicacion, viajes_en_ejecucion
6  from routers.viajes import listar_viajes, obtener_viaje
7
8  def _mock_viaje(v_id=1, codigo="VJ-01", estado="EN_EJECUCION", horas_retraso=0, t_nom="Carlos Pérez"):
9      v = Mock(id=v_id, codigo=codigo, origen="UIO", destino="GYE", horas_retraso=horas_retraso,
10             transportista_id=10)
11      v.latitud_origen, v.longitud_origen, v.latitud_destino, v.longitud_destino = -0.18, -78.46,
12      -2.17, -79.92
13      v.latitud_actual, v.longitud_actual = -1.25, -79.20
14      v.estado = Mock(value=estado)
15      v.estado.__eq__ = lambda self, other: self.value == (other.value if hasattr(other, 'value')
16      else other)
17      v.transportista = Mock(usuario=Mock(nombres=t_nom), placa_vehiculo="ABC-1234") if t_nom else
18      None
19      return v
20
21 class TestGestionarViajes:
22     """Pruebas unitarias optimizadas para el monitoreo de mapas y gestión de viajes."""
23
24     # -- PATRÓN STRATEGY: UBICACIÓN GPS --
25     @patch("routers.monitoreo.GPSTrackerContext")
26     @patch("routers.monitoreo.SimulatedInterpolationStrategy")
27     def test_ubicacion_strategy_simulada(self, mock_strat, mock_ctx):
28         mock_ctx.return_value.execute.return_value = {"lat": -1.05, "lng": -79.10, "tipo_gps": "
29 Simulado"}
30         res = obtener_ubicacion(1, "simulado", db=Mock(query=Mock(return_value=Mock(filter=Mock(
31         return_value=Mock(first=Mock(return_value=_mock_viaje()))))), current_user=Mock(rol="
32 SECRETARIA"))
33         assert res["lat"] == -1.05 and "Simulado" in res["tipo_gps"]
34
35     @patch("routers.monitoreo.GPSTrackerContext")
36     @patch("routers.monitoreo.RealGPSTrackingStrategy")
37     def test_ubicacion_strategy_real(self, mock_strat, mock_ctx):
38         mock_ctx.return_value.execute.return_value = {"lat": -0.95, "lng": -78.80, "tipo_gps": "
39 GPS Real"}
40         res = obtener_ubicacion(2, "real", db=Mock(query=Mock(return_value=Mock(filter=Mock(
41         return_value=Mock(first=Mock(return_value=_mock_viaje(2))))))), current_user=Mock(rol="
42 COORDINADOR"))
43         assert res["lat"] == -0.95 and "Real" in res["tipo_gps"]
44
45     def test_ubicacion_viaje_no_encontrado(self):
46         db = Mock(query=Mock(return_value=Mock(filter=Mock(return_value=Mock(first=Mock(
47         return_value=None))))))
48         with pytest.raises(HTTPException) as exc:
49             obtener_ubicacion(999, "simulado", db=db, current_user=Mock(rol="SECRETARIA"))
50         assert exc.value.status_code == 404
51
52     # -- PATRÓN DECORATOR: VIAJES EN EJECUCIÓN (VISTA MAPA) --
53     def test_viajes_en_ejecucion_con_datos(self):
54         db = Mock(query=Mock(return_value=Mock(filter=Mock(return_value=Mock(order_by=Mock(
55         return_value=Mock(all=Mock(return_value=[_mock_viaje(1, "VJ-01"), _mock_viaje(2, "VJ-02", "
56 TRANSPORTISTA_ASIGNADO"))]))))))))
57         res = viajes_en_ejecucion(db=db, current_user=Mock(rol="COORDINADOR"))
58         assert len(res) == 2 and res[0]["latitud_actual"] == -1.25 and "decoracion_alerta" in
59         res[0]

```

```

47 def test_viajes_en_ejecucion_sin_viajes_activos(self):
48     db = Mock(query=Mock(return_value=Mock(filter=Mock(return_value=Mock(order_by=Mock(
return_value=Mock(all=Mock(return_value=[]))))))))))
49     assert len(viajes_en_ejecucion(db=db, current_user=Mock(rol="COORDINADOR"))) == 0
50
51 def test_viaje_con_retraso_muestra_alerta_alto_riesgo(self):
52     db = Mock(query=Mock(return_value=Mock(filter=Mock(return_value=Mock(order_by=Mock(
return_value=Mock(all=Mock(return_value=[_mock_viaje(horas_retraso=3.5)]))))))))))
53     assert viajes_en_ejecucion(db=db, current_user=Mock(rol="COORDINADOR"))[0]["
decoracion_alerta"] == "ALTO RIESGO"
54
55 def test_viaje_sin_retraso_muestra_operacion_normal(self):
56     db = Mock(query=Mock(return_value=Mock(filter=Mock(return_value=Mock(order_by=Mock(
return_value=Mock(all=Mock(return_value=[_mock_viaje(horas_retraso=0)]))))))))))
57     assert viajes_en_ejecucion(db=db, current_user=Mock(rol="COORDINADOR"))[0]["
decoracion_alerta"] == "OPERACIÓN NORMAL"
58
59 # -- OBTENER Y LISTAR VIAJES (INTEGRIDAD CARTOGRÁFICA) --
60 def test_obtener_viaje_con_coordenadas_gps(self):
61     db = Mock(query=Mock(return_value=Mock(filter=Mock(return_value=Mock(first=Mock(
return_value=_mock_viaje(10)))))))
62     res = obtener_viaje(10, db=db, current_user=Mock(rol="SECRETARIA"))
63     assert res["latitud_origen"] == -0.18 and res["latitud_actual"] == -1.25
64
65 def test_obtener_viaje_sin_transportista(self):
66     db = Mock(query=Mock(return_value=Mock(filter=Mock(return_value=Mock(first=Mock(
return_value=_mock_viaje(20, estado="DISPONIBLE", t_nom=None)))))))
67     res = obtener_viaje(20, db=db, current_user=Mock(rol="SECRETARIA"))
68     assert res["latitud_origen"] is not None and res["transportista_nombres"] is None
69
70 def test_listar_viajes_retorna_coordenadas(self):
71     db = Mock(query=Mock(return_value=Mock(order_by=Mock(return_value=Mock(all=Mock(
return_value=[_mock_viaje(1), _mock_viaje(2)])))))))
72     u = Mock(rol=Mock(value="SECRETARIA"))
73     res = listar_viajes(db=db, current_user=u)
74     assert len(res) == 2 and "latitud_actual" in res[0]

```

Salida en Consola

```

tests/test_gestionar_viajes.py::TestGestionarViajes::test_ubicacion_strategy_simulada PASSED
tests/test_gestionar_viajes.py::TestGestionarViajes::test_ubicacion_strategy_real PASSED
tests/test_gestionar_viajes.py::TestGestionarViajes::test_ubicacion_viaje_no_encontrado PASSED
tests/test_gestionar_viajes.py::TestGestionarViajes::test_viajes_en_ejecucion_con_datos PASSED
tests/test_gestionar_viajes.py::TestGestionarViajes::test_viajes_en_ejecucion_sin_viajes_activos PASSED
tests/test_gestionar_viajes.py::TestGestionarViajes::test_viaje_con_retraso_muestra_alerta_alto_riesgo
PASSED
tests/test_gestionar_viajes.py::TestGestionarViajes::test_viaje_sin_retraso_muestra_operacion_normal
PASSED
tests/test_gestionar_viajes.py::TestGestionarViajes::test_obtener_viaje_con_coordenadas_gps PASSED
tests/test_gestionar_viajes.py::TestGestionarViajes::test_obtener_viaje_sin_transportista PASSED
tests/test_gestionar_viajes.py::TestGestionarViajes::test_listar_viajes_retorna_coordenadas PASSED

===== 10 passed in 0.18s =====

```

4.4. RF004: Monitorear Viajes

Descripción: Controlar y supervisar los viajes de transporte detectando imprevistos para poder tomar acciones.

- **Archivo Test:** test_monitorear_viajes.py

- **Total de Pruebas:** 5 pruebas
- **Estado:** 100 % PASSED

Implementación

```

1  # tests/test_monitorear_viajes.py
2  from unittest.mock import Mock, patch
3  import pytest
4  from fastapi import HTTPException
5  from routers.monitoreo import modificar_ruta, ver_log_notificaciones
6  from schemas.viaje_schemas import ModificarRutaRequest
7  from models.models import EstadoViajeEnum
8
9  class TestMonitorearViajes:
10     """Pruebas unitarias optimizadas para el control de imprevistos y logs de notificación."""
11
12     # -- GESTIÓN DE IMPREVISTOS Y NOTIFICACIÓN (REQ004) --
13     @patch("patterns.adapter.city_gps_adapter.CityToGPSAdapter.get_coordinates")
14     @patch("patterns.observer.trip_observer.TripSubject.notify_all")
15     @patch("routers.monitoreo._viaje_monitoreo")
16     def test_modificar_ruta_exitoso_notifica_transportista(self, mock_out, mock_notify, mock_gps):
17         viaje = Mock(id=1, codigo="VJ-TEST", estado=EstadoViajeEnum.EN_EJECUCION)
18         db = Mock(query=Mock(return_value=Mock(filter=Mock(return_value=Mock(first=Mock(
19             return_value=viaje))))))
20         body = ModificarRutaRequest(nueva_ruta_json="GUAYAQUIL_SUR", motivo="Vía cerrada por
21             deslave (imprevisto)")
22         mock_gps.return_value, mock_out.return_value = {"lat": -2.2, "lng": -79.9}, {"id": 1, "
23             estado": "EN_EJECUCION"}
24
25         res = modificar_ruta(viaje_id=1, body=body, db=db, current_user=Mock(id=100, rol="
26             COORDINADOR"))
27
28         assert viaje.ruta_json == "GUAYAQUIL_SUR" and viaje.latitud_destino == -2.2
29         mock_notify.assert_called_once()
30         assert "Desvío registrado" in mock_notify.call_args[1]["extra_info"]
31         assert res == {"id": 1, "estado": "EN_EJECUCION"}
32
33     def test_modificar_ruta_viaje_no_encontrado(self):
34         db = Mock(query=Mock(return_value=Mock(filter=Mock(return_value=Mock(first=Mock(
35             return_value=None))))))
36         with pytest.raises(HTTPException) as exc:
37             modificar_ruta(99, ModificarRutaRequest(nueva_ruta_json="X", motivo="Y"), db, Mock(
38                 rol="COORDINADOR"))
39         assert exc.value.status_code == 404
40
41     def test_modificar_ruta_viaje_no_ejecucion(self):
42         viaje = Mock(id=1, estado=EstadoViajeEnum.COMPLETADO)
43         db = Mock(query=Mock(return_value=Mock(filter=Mock(return_value=Mock(first=Mock(
44             return_value=viaje))))))
45         with pytest.raises(HTTPException) as exc:
46             modificar_ruta(1, ModificarRutaRequest(nueva_ruta_json="X", motivo="Y"), db, Mock(
47                 rol="COORDINADOR"))
48         assert exc.value.status_code == 409
49
50     def test_modificar_ruta_vacia(self):
51         viaje = Mock(id=1, estado=EstadoViajeEnum.EN_EJECUCION)
52         db = Mock(query=Mock(return_value=Mock(filter=Mock(return_value=Mock(first=Mock(
53             return_value=viaje))))))
54         with pytest.raises(HTTPException) as exc:
55             modificar_ruta(1, ModificarRutaRequest(nueva_ruta_json="", motivo="Y"), db, Mock(rol
56                 ="COORDINADOR"))

```

```

47     assert exc.value.status_code == 400
48
49     # -- HISTORIAL / AUDITORÍA DE LOGS --
50     def test_ver_log_notificaciones(self):
51         logs = ["[10:00] Push enviada. Evento: MODIFICAR_RUTA.", "[11:00] SMS enviada. Evento:
ALERTA."]
52         with patch("routers.monitoreo.global_notification_observer.envios", logs):
53             res = ver_log_notificaciones(current_user=Mock(rol="COORDINADOR"))
54             assert len(res) == 2 and res == logs

```

Salida en Consola

```

● ● ●
tests/
test_monitorear_viajes.py::TestMonitorearViajes::test_modificar_ruta_exitoso_notifica_transportista
PASSED
tests/test_monitorear_viajes.py::TestMonitorearViajes::test_modificar_ruta_viaje_no_encontrado PASSED
tests/test_monitorear_viajes.py::TestMonitorearViajes::test_modificar_ruta_viaje_no_ejecucion PASSED
tests/test_monitorear_viajes.py::TestMonitorearViajes::test_modificar_ruta_vacia PASSED
tests/test_monitorear_viajes.py::TestMonitorearViajes::test_ver_log_notificaciones PASSED

===== 5 passed in 0.10s =====

```

4.5. RF005: Crear Transportista

Descripción: Registrar un nuevo transportista con la información requerida.

- Archivo Test: test_crear_transportista.py
- Total de Pruebas: 3 pruebas
- Estado: 100 % PASSED

Implementación

```

1  # tests/test_crear_transportista.py
2  from unittest.mock import Mock, patch
3  import pytest
4  from fastapi import HTTPException
5  from routers.transportistas import crear_transportista
6  from models.models import Usuario
7
8  class TestCrearTransportista:
9      """Pruebas unitarias optimizadas para el REQ005: Registrar transportistas."""
10
11      @patch("core.security.hash_password", return_value="hashed_123")
12      @patch("routers.transportistas._build_out", return_value={"id": 1, "nombres": "Juan Pérez"})
13      @patch("routers.transportistas.registrar_auditoria")
14      def test_crear_transportista_exitoso(self, mock_auditoria, mock_out, mock_hash):
15          db = Mock()
16          db.query.return_value.filter.return_value.first.return_value = None
17          req = Mock()
18          req.client.host = "127.0.0.1"
19
20          res = crear_transportista("1234567890", "Juan Pérez", "juan@correo.com", "clave", "ABC
-1234", "Camión", 5.5, "Quito", "099", db, Mock(rol="COORDINADOR"), req)
21
22          assert res == {"id": 1, "nombres": "Juan Pérez"}
23          assert db.add.call_count == 2 and db.commit.called

```

```

24     mock_auditoria.assert_called_once()
25
26     def test_crear_transportista_correo_existente(self):
27         db = Mock()
28         db.query.return_value.filter.return_value.first.return_value = Usuario(id=1)
29
30         with pytest.raises(HTTPException) as exc:
31             crear_transportista("1234567890", "Juan", "existente@correo.com", "clv", None, None,
32                                 None, None, None, db, Mock(rol="COORDINADOR"))
33             assert exc.value.status_code == 400 and "correo ya está registrado" in exc.value.detail.lower()
34
35     def test_crear_transportista_cedula_existente(self):
36         db = Mock()
37         db.query.return_value.filter.return_value.first.side_effect = [None, Usuario(id=1)]
38
39         with pytest.raises(HTTPException) as exc:
40             crear_transportista("0999999999", "Juan", "nuevo@correo.com", "clv", None, None,
41                                 None, None, None, db, Mock(rol="COORDINADOR"))
42             assert exc.value.status_code == 400 and "cédula ya está registrada" in exc.value.detail.lower()

```

Salida en Consola

```

tests/test_crear_transportista.py::TestCrearTransportista::test_crear_transportista_exitoso PASSED
tests/test_crear_transportista.py::TestCrearTransportista::test_crear_transportista_correo_existente PASSED
tests/test_crear_transportista.py::TestCrearTransportista::test_crear_transportista_cedula_existente PASSED

===== 3 passed in 0.08s =====

```

4.6. RF006: Ingresar Documentacion

Descripción: Cargar documentación laboral de los transportistas

- **Archivo Test:** test_ingresar_documentacion.py
- **Total de Pruebas:** 5 pruebas
- **Estado:** 100 % PASSED

Implementación

```

1  # tests/test_ingresar_documentacion.py
2  from unittest.mock import AsyncMock, Mock, patch
3  import pytest
4  from fastapi import HTTPException, UploadFile
5  from routers.transportistas import importar_documento
6  from models.models import Transportista, Documento, EstadoDocEnum
7
8  def _mock_upload_file(c_type="application/pdf", content=b"%PDF-1.4", filename="doc.pdf"):
9      mock_file = AsyncMock(spec=UploadFile, content_type=c_type, filename=filename)
10     mock_file.read.return_value = content
11     return mock_file
12
13 def _mock_user(u_id=10):
14     u = Mock(id=u_id)

```

```

15     u.rol = Mock(value="TRANSPORTISTA")
16     return u
17
18 @pytest.mark.anyio
19 class TestIngresarDocumentacion:
20     """Pruebas unitarias asíncronas optimizadas para el REQ006: Ingresar documentación."""
21
22     @patch("routers.transportistas.settings")
23     @patch("routers.transportistas.registrar_auditoria")
24     async def test_importar_documento_exitoso(self, mock_auditoria, mock_settings):
25         mock_settings.MAX_FILE_SIZE_MB, db = 10, Mock()
26         db.query.return_value.filter.return_value.first.side_effect = [Mock(spec=Transportista,
27 id=5, usuario_id=10), None]
27         req = Mock()
28         req.client.host = "127.0.0.1"
29
30         res = await importar_documento(5, "LICENCIA_E", "2028-01-01", _mock_upload_file(content=
31 b"fake-bytes"), db, _mock_user(10), req)
32
33         assert "correctamente" in res["mensaje"].lower() and db.add.called and db.commit.called
34         doc = db.add.call_args[0][0]
35         assert isinstance(doc, Documento) and doc.estado == EstadoDocEnum.PENDIENTE and doc.
36 contenido_pdf == b"fake-bytes"
37         mock_auditoria.assert_called_once()
38
39     @patch("routers.transportistas.settings")
40     async def test_importar_documento_otro_transportista_denegado(self, mock_settings):
41         db = Mock()
42         db.query.return_value.filter.return_value.first.return_value = Mock(spec=Transportista,
43 id=5, usuario_id=99)
44         with pytest.raises(HTTPException) as exc:
45             await importar_documento(5, "CEDULA", None, _mock_upload_file(), db, _mock_user(10))
46         assert exc.value.status_code == 403 and "no tiene permiso" in exc.value.detail.lower()
47
48     @patch("routers.transportistas.settings")
49     async def test_importar_documento_tipo_invalido(self, mock_settings):
50         db = Mock()
51         db.query.return_value.filter.return_value.first.return_value = Mock(spec=Transportista,
52 id=5, usuario_id=10)
53         with pytest.raises(HTTPException) as exc:
54             await importar_documento(5, "DIPLOMA_FALSO", None, _mock_upload_file(), db,
55 _mock_user(10))
56         assert exc.value.status_code == 400 and "tipo inválido" in exc.value.detail.lower()
57
58     @patch("routers.transportistas.settings")
59     async def test_importar_documento_no_es_pdf(self, mock_settings):
60         db = Mock()
61         db.query.return_value.filter.return_value.first.return_value = Mock(spec=Transportista,
62 id=5, usuario_id=10)
63         with pytest.raises(HTTPException) as exc:
64             await importar_documento(5, "CEDULA", None, _mock_upload_file("image/png", filename=
65 "f.png"), db, _mock_user(10))
66         assert exc.value.status_code == 400 and "solo se aceptan archivos en formato pdf" in exc
67 .value.detail.lower()
68
69     @patch("routers.transportistas.settings")
70     async def test_importar_documento_excede_limite_peso(self, mock_settings):
71         mock_settings.MAX_FILE_SIZE_MB, db = 1, Mock()
72         db.query.return_value.filter.return_value.first.return_value = Mock(spec=Transportista,
73 id=5, usuario_id=10)
74         with pytest.raises(HTTPException) as exc:
75             await importar_documento(5, "CEDULA", None, _mock_upload_file(content=b"0" * (2 *
76 1024 * 1024)), db, _mock_user(10))

```



```
67 assert exc.value.status_code == 400 and "excede el límite" in exc.value.detail.lower()
```

Salida en Consola

```
tests/test_ingresar_documentacion.py::TestIngresarDocumentacion::test_importar_documento_exitoso PASSED
tests/
test_ingresar_documentacion.py::TestIngresarDocumentacion::test_importar_documento_otro_transportista_d
enegado PASSED
tests/test_ingresar_documentacion.py::TestIngresarDocumentacion::test_importar_documento_tipo_invalido
PASSED
tests/test_ingresar_documentacion.py::TestIngresarDocumentacion::test_importar_documento_no_es_pdf
PASSED
tests/
test_ingresar_documentacion.py::TestIngresarDocumentacion::test_importar_documento_excede_limite_peso
PASSED

===== 5 passed in 0.15s =====
```

4.7. RF007: Verificar Documentación

Descripción: Validar documentación laboral de los transportistas.

- **Archivo Test:** test_verificar_documentacion.py
- **Total de Pruebas:** 5 pruebas
- **Estado:** 100 % PASSED

Implementación

```
1 # tests/test_verificar_documentacion.py
2 from unittest.mock import Mock, patch
3 from datetime import datetime
4 import pytest
5 from fastapi import HTTPException
6 from routers.transportistas import revisar_documento
7 from schemas.transportista_schemas import RevisionDocumentoRequest
8 from models.models import Documento, EstadoDocEnum
9
10 class TestVerificarDocumentacion:
11     """Pruebas unitarias optimizadas para el REQ007: Validar documentación laboral."""
12
13     @patch("routers.transportistas.registrar_auditoria")
14     def test_revisar_documento_aprobado(self, mock_auditoria):
15         doc, db, req = Mock(spec=Documento, id=1, estado=EstadoDocEnum.PENDIENTE), Mock(), Mock()
16
17         db.query.return_value.filter.return_value.first.return_value = doc
18         req.client.host = "127.0.0.1"
19         body = RevisionDocumentoRequest(estado="APROBADO", observacion="OK", fecha_vencimiento=
20             datetime(2028, 1, 1))
21
22         res = revisar_documento(10, 1, body, db, Mock(id=50, rol="SECRETARIA"), req)
23
24         assert "aprobado correctamente" in res["mensaje"].lower() and doc.estado == "APROBADO"
25         assert doc.revisado_por_id == 50 and doc.fecha_vencimiento == datetime(2028, 1, 1)
26         db.commit.assert_called_once()
27         mock_auditoria.assert_called_once()
```



```

27 @patch("routers.transportistas.registrar_auditoria")
28 def test_revisar_documento_rechazado_con_observacion(self, mock_auditoria):
29     doc, db = Mock(spec=Documento, estado=EstadoDocEnum.PENDIENTE), Mock()
30     db.query.return_value.filter.return_value.first.return_value = doc
31     body = RevisionDocumentoRequest(estado="RECHAZADO", observacion="Ilegible")
32
33     res = revisar_documento(10, 1, body, db, Mock(id=50, rol="SECRETARIA"), Mock())
34
35     assert "rechazado correctamente" in res["mensaje"].lower()
36     assert doc.estado == "RECHAZADO" and doc.observacion == "Ilegible"
37     db.commit.assert_called_once()
38
39 def test_revisar_documento_rechazado_sin_observacion(self):
40     db = Mock(query=Mock(return_value=Mock(filter=Mock(return_value=Mock(first=Mock(
41     return_value=Mock(spec=Documento)))))))
42     with pytest.raises(HTTPException) as exc:
43         revisar_documento(10, 1, RevisionDocumentoRequest(estado="RECHAZADO", observacion=""),
44         db, Mock(rol="SECRETARIA"))
45     assert exc.value.status_code == 400 and "observación" in exc.value.detail.lower()
46
47 def test_revisar_documento_estado_invalido(self):
48     db = Mock(query=Mock(return_value=Mock(filter=Mock(return_value=Mock(first=Mock(
49     return_value=Mock(spec=Documento)))))))
50     with pytest.raises(HTTPException) as exc:
51         revisar_documento(10, 1, RevisionDocumentoRequest(estado="OTRO_ESTADO"), db, Mock(
52         rol="SECRETARIA"))
53     assert exc.value.status_code == 400 and "aprobado o rechazado" in exc.value.detail.lower()
54
55 def test_revisar_documento_no_encontrado(self):
56     db = Mock(query=Mock(return_value=Mock(filter=Mock(return_value=Mock(first=Mock(
57     return_value=None)))))))
58     with pytest.raises(HTTPException) as exc:
59         revisar_documento(10, 999, RevisionDocumentoRequest(estado="APROBADO"), db, Mock(rol=
60         "SECRETARIA"))
61     assert exc.value.status_code == 404

```

Salida en Consola

```

tests/test_verificar_documentacion.py::TestVerificarDocumentacion::test_revisar_documento_aprobado
PASSED
tests/
test_verificar_documentacion.py::TestVerificarDocumentacion::test_revisar_documento_rechazado_con_obser
vacion PASSED
tests/
test_verificar_documentacion.py::TestVerificarDocumentacion::test_revisar_documento_rechazado_sin_obser
vacion PASSED
tests/
test_verificar_documentacion.py::TestVerificarDocumentacion::test_revisar_documento_estado_invalido
PASSED
tests/test_verificar_documentacion.py::TestVerificarDocumentacion::test_revisar_documento_no_encontrado
PASSED
===== 5 passed in 0.09s =====

```

4.8. RF008: Recuperar Contraseña

Descripción: Módulo para recuperar contraseñas y volver a obtener acceso al sistema.

- Archivo Test: test_recuperar_contraseña.py

- Total de Pruebas: 4 pruebas
- Estado: 100 % PASSED

Implementación

```

1  # tests/test_recuperar_contrasena.py
2  from unittest.mock import Mock, patch
3  import pytest
4  from fastapi import HTTPException
5  from routers.auth import solicitar_recuperacion
6  from schemas.auth_schemas import RecuperarPasswordRequest
7  from models.models import Usuario
8
9  @patch("routers.auth.enviar_nueva_contrasena")
10 @patch("routers.auth.generar_contrasena")
11 class TestRecuperarContrasena:
12     """Pruebas unitarias optimizadas para el REQ008: Recuperación de contraseñas."""
13
14     @patch("routers.auth.hash_password", return_value="hashed_Clave123")
15     def test_solicitar_recuperacion_exitoso(self, mock_hash, mock_gen, mock_env):
16         mock_gen.return_value, db = "Clave123", Mock()
17         user = Mock(spec=Usuario, correo="test@correo.com", activo=True)
18         db.query.return_value.filter.return_value.first.return_value = user
19
20         res = solicitar_recuperacion(RecuperarPasswordRequest(correo="test@correo.com"), db)
21
22         assert "nueva contraseña" in res["mensaje"].lower() and user.hashed_password == "
hashed_Clave123"
23         db.commit.assert_called_once()
24         mock_env.assert_called_once_with("test@correo.com", "Clave123")
25
26     def test_solicitar_recuperacion_correo_inexistente(self, mock_gen, mock_env):
27         db = Mock(query=Mock(return_value=Mock(filter=Mock(return_value=Mock(first=Mock(
return_value=None))))))
28         res = solicitar_recuperacion(RecuperarPasswordRequest(correo="no@correo.com"), db)
29         assert "nueva contraseña" in res["mensaje"].lower()
30         mock_gen.assert_not_called()
31
32     def test_solicitar_recuperacion_usuario_inactivo(self, mock_gen, mock_env):
33         db = Mock()
34         db.query.return_value.filter.return_value.first.return_value = Mock(spec=Usuario, correo
="t@c.com", activo=False)
35         res = solicitar_recuperacion(RecuperarPasswordRequest(correo="t@c.com"), db)
36         assert "nueva contraseña" in res["mensaje"].lower()
37         mock_gen.assert_not_called()
38
39     def test_solicitar_recuperacion_falla_envio_correo(self, mock_gen, mock_env):
40         mock_gen.return_value, mock_env.side_effect = "Clave123", Exception("SMTP error")
41         db = Mock(query=Mock(return_value=Mock(filter=Mock(return_value=Mock(first=Mock(
return_value=Mock(spec=Usuario, correo="t@c.com", activo=True)))))))
42
43         with pytest.raises(HTTPException) as exc:
44             solicitar_recuperacion(RecuperarPasswordRequest(correo="t@c.com"), db)
45         assert exc.value.status_code == 500 and "enviar el correo" in exc.value.detail.lower()

```

Salida en Consola

```

tests/test_recuperar_contrasena.py::TestRecuperarContrasena::test_solicitar_recuperacion_exitoso PASSED
tests/
test_recuperar_contrasena.py::TestRecuperarContrasena::test_solicitar_recuperacion_correo_inexistente
PASSED
tests/
test_recuperar_contrasena.py::TestRecuperarContrasena::test_solicitar_recuperacion_usuario_inactivo
PASSED
tests/
test_recuperar_contrasena.py::TestRecuperarContrasena::test_solicitar_recuperacion_falla_envio_correo
PASSED

===== 4 passed in 0.08s =====

```

4.9. RF009: Editar Transportista

Descripción: Actualizar la información del vehículo del transportista.

- **Archivo Test:** test_editar_transportista.py
- **Total de Pruebas:** 4 pruebas
- **Estado:** 100 % PASSED

Implementación

```

1  # tests/test_editar_transportista.py
2  from unittest.mock import Mock, patch
3  import pytest
4  from fastapi import HTTPException
5  from routers.transportistas import editar_transportista
6  from schemas.transportista_schemas import TransportistaUpdate
7  from models.models import Transportista, Usuario
8
9  class TestEditarTransportista:
10     """Pruebas unitarias optimizadas para el REQ009: Editar transportistas."""
11
12     @patch("routers.transportistas.registrar_auditoria")
13     @patch("routers.transportistas._build_out", return_value={"id": 1})
14     def test_editar_transportista_vehiculo_exitoso(self, mock_out, mock_auditoria):
15         t, db, req = Mock(spec=Transportista, placa_vehiculo="A", tipo_vehiculo="B",
16         capacidad_ton=1.0), Mock(), Mock()
17         t.usuario = Mock(spec=Usuario, id=10, nombres="Juan")
18         db.query.return_value.filter.return_value.first.return_value = t
19         req.client.host = "127.0.0.1"
20         body = TransportistaUpdate(placa_vehiculo="NUEVA-123", tipo_vehiculo="Camión",
21         capacidad_ton=10.5)
22
23         res = editar_transportista(1, body, db, Mock(rol="COORDINADOR"), req)
24
25         assert t.placa_vehiculo == "NUEVA-123" and t.capacidad_ton == 10.5
26         db.commit.assert_called_once()
27         mock_auditoria.assert_called_once()
28         assert res == {"id": 1}
29
30     @patch("routers.transportistas.registrar_auditoria")
31     @patch("routers.transportistas._build_out", return_value={"id": 1})
32     def test_editar_transportista_nombres_correo_exitoso(self, mock_out, mock_auditoria):
33         t, db = Mock(spec=Transportista), Mock()

```

```

32     t.usuario = Mock(spec=Usuario, id=10, nombres="Old", correo="old@c.com")
33     db.query.return_value.filter.return_value.first.side_effect = [t, None]
34     body = TransportistaUpdate(nombres="Juan Nuevo", correo="nuevo@correo.com")
35
36     editar_transportista(1, body, db, Mock(rol="COORDINADOR"))
37
38     assert t.usuario.nombres == "Juan Nuevo" and t.usuario.correo == "nuevo@correo.com"
39     db.commit.assert_called_once()
40
41     def test_editar_transportista_correo_duplicado(self):
42         t, db = Mock(spec=Transportista), Mock()
43         t.usuario = Mock(spec=Usuario, id=10, correo="juan@c.com")
44         db.query.return_value.filter.return_value.first.side_effect = [t, Mock(spec=Usuario, id
45 =99)]
46
47         with pytest.raises(HTTPException) as exc:
48             editar_transportista(1, TransportistaUpdate(correo="ocupado@c.com"), db, Mock(rol="
COORDINADOR"))
49         assert exc.value.status_code == 400 and "ya se encuentra registrado" in exc.value.detail
50         .lower()
51
52     def test_editar_transportista_no_encontrado(self):
53         db = Mock(query=Mock(return_value=Mock(filter=Mock(return_value=Mock(first=Mock(
54 return_value=None))))))
55         with pytest.raises(HTTPException) as exc:
56             editar_transportista(999, TransportistaUpdate(placa_vehiculo="NUEVA"), db, Mock(rol=
"COORDINADOR"))
57         assert exc.value.status_code == 404

```

Salida en Consola

```

● ● ●
tests/test_editar_transportista.py::TestEditarTransportista::test_editar_transportista_vehiculo_exitoso
PASSED
tests/
test_editar_transportista.py::TestEditarTransportista::test_editar_transportista_nombres_correo_exitoso
PASSED
tests/test_editar_transportista.py::TestEditarTransportista::test_editar_transportista_correo_duplicado
PASSED
tests/test_editar_transportista.py::TestEditarTransportista::test_editar_transportista_no_encontrado
PASSED

===== 4 passed in 0.08s =====

```

4.10. RF010: Eliminar Transportista

Descripción: Eliminar transportistas del sistema.

- **Archivo Test:** test_eliminar_transportista.py
- **Total de Pruebas:** 4 pruebas
- **Estado:** 100 % PASSED

Implementación

```

1 # tests/test_eliminar_transportista.py
2 from unittest.mock import Mock, patch
3 import pytest

```

```

4 from fastapi import HTTPException
5 from routers.transportistas import desactivar_transportista, eliminar_permanentemente
6 from schemas.transportista_schemas import EliminarTransportistaRequest
7 from models.models import Transportista, Usuario
8
9 class TestEliminarTransportista:
10     """Pruebas unitarias optimizadas para el REQ010: Eliminar transportistas."""
11
12     @patch("routers.transportistas.registrar_auditoria")
13     def test_desactivar_transportista_exitoso(self, mock_auditoria):
14         t, db, req = Mock(spec=Transportista, usuario=Mock(spec=Usuario, id=10, activo=True)),
15         Mock(), Mock()
16         db.query.return_value.filter.return_value.first.side_effect = [t, None]
17         req.client.host = "127.0.0.1"
18         body = EliminarTransportistaRequest(razon="Fin contrato", observaciones="OK")
19
20         res = desactivar_transportista(1, body, db, Mock(rol="COORDINADOR"), req)
21
22         assert "desactivado correctamente" in res["mensaje"].lower() and t.usuario.activo is
23         False
24         db.commit.assert_called_once()
25         mock_auditoria.assert_called_once()
26
27     def test_desactivar_transportista_con_viajes_activos(self):
28         t, db = Mock(spec=Transportista, usuario=Mock(spec=Usuario, id=10, activo=True)), Mock()
29         db.query.return_value.filter.return_value.first.side_effect = [t, Mock()]
30
31         with pytest.raises(HTTPException) as exc:
32             desactivar_transportista(1, EliminarTransportistaRequest(razon="Baja"), db, Mock(rol=
33             "COORDINADOR"))
34         assert exc.value.status_code == 409 and "viajes en ejecución" in exc.value.detail.lower
35         ()
36         db.commit.assert_not_called()
37
38     def test_desactivar_transportista_no_encontrado(self):
39         db = Mock(query=Mock(return_value=Mock(filter=Mock(return_value=Mock(first=Mock(
40         return_value=None))))))
41         with pytest.raises(HTTPException) as exc:
42             desactivar_transportista(999, EliminarTransportistaRequest(razon="Error"), db, Mock(
43             rol="COORDINADOR"))
44         assert exc.value.status_code == 404
45
46     @patch("routers.transportistas.registrar_auditoria")
47     def test_eliminar_permanentemente_exitoso(self, mock_auditoria):
48         t, db, req = Mock(spec=Transportista, usuario=Mock(spec=Usuario, id=10)), Mock(), Mock()
49         db.query.return_value.filter.return_value.first.side_effect = [t, None]
50         req.client.host = "127.0.0.1"
51
52         res = eliminar_permanentemente(1, db, Mock(rol="COORDINADOR"), req)
53
54         assert "eliminado permanentemente" in res["mensaje"].lower()
55         db.delete.assert_any_call(t)
56         db.delete.assert_any_call(t.usuario)
57         db.commit.assert_called_once()
58         mock_auditoria.assert_called_once()

```

Salida en Consola

```

tests/test_eliminar_transportista.py::TestEliminarTransportista::test_desactivar_transportista_exitoso
PASSED
tests/
test_eliminar_transportista.py::TestEliminarTransportista::test_desactivar_transportista_con_viajes_act
ivos PASSED
tests/
test_eliminar_transportista.py::TestEliminarTransportista::test_desactivar_transportista_no_encontrado
PASSED
tests/test_eliminar_transportista.py::TestEliminarTransportista::test_eliminar_permanentemente_exitoso
PASSED

===== 4 passed in 0.08s =====

```

4.11. RF011: Crear Viajes

Descripción: Registrar nuevos viajes.

- **Archivo Test:** test_crear_viajes.py
- **Total de Pruebas:** 3 pruebas
- **Estado:** 100 % PASSED

Implementación

```

1  # tests/test_crear_viajes.py
2  from unittest.mock import Mock, patch
3  import pytest
4  from fastapi import HTTPException
5  from routers.viajes import crear_viaje
6  from schemas.viaje_schemas import ViajeCreate
7  from models.models import Viaje, EstadoViajeEnum
8
9  class TestCrearViajes:
10     """Pruebas unitarias optimizadas para el REQ011: Crear Viajes."""
11
12     @patch("routers.viajes.registrar_auditoria")
13     @patch("routers.viajes._generar_codigo", return_value="VJ-TEST-001")
14     @patch("routers.viajes.CityToGPSAdapter.get_coordinates")
15     @patch("routers.viajes._viaje_out", return_value={"id": 1, "codigo": "VJ-TEST-001"})
16     def test_crear_viaje_exitoso(self, mock_out, mock_gps, mock_cod, mock_auditoria):
17         db, req = Mock(), Mock()
18         req.client.host = "127.0.0.1"
19         mock_gps.side_effect = [{"lat": -0.2, "lng": -78.5}, {"lat": -2.2, "lng": -79.9}]
20         body = ViajeCreate(tipo_mercancia="Electrónicos", peso_total_kg=5000, dimensiones="10
x5x3", numero_contenedor="MSCU1234567", origen="QUITO", destino="GUAYAQUIL", punto_recepcion
="B1", destinatario_nombre="XYZ")
21
22         res = crear_viaje(body, db, Mock(id=50, rol="SECRETARIA"), req)
23
24         assert res == {"id": 1, "codigo": "VJ-TEST-001"}
25         db.add.assert_called_once()
26         viaje = db.add.call_args[0][0]
27         assert isinstance(viaje, Viaje) and viaje.estado == EstadoViajeEnum.DISPONIBLE and viaje
.latitud_actual == -0.2
28         db.commit.assert_called_once()
29         mock_auditoria.assert_called_once()
30

```

```

31 def test_crear_viaje_contenedor_invalido(self):
32     body = ViajeCreate(tipo_mercancia="Carga", peso_total_kg=1000, numero_contenedor="MAL123",
33     origen="QUITO", destino="CUENCA", punto_recepcion="B1", destinatario_nombre="D")
34     with pytest.raises(HTTPException) as exc:
35         crear_viaje(body, Mock(), Mock(rol="SECRETARIA"))
36     assert exc.value.status_code == 400 and "contenedor inválido" in exc.value.detail.lower()
37
38 def test_crear_viaje_origen_destino_iguales(self):
39     body = ViajeCreate(tipo_mercancia="Carga", peso_total_kg=1000, origen=" Quito ",
40     destino="QUITO", punto_recepcion="B1", destinatario_nombre="D")
41     with pytest.raises(HTTPException) as exc:
42         crear_viaje(body, Mock(), Mock(rol="SECRETARIA"))
43     assert exc.value.status_code == 400 and "iguales" in exc.value.detail.lower()

```

Salida en Consola

```

tests/test_crear_viajes.py::TestCrearViajes::test_crear_viaje_exitoso PASSED
tests/test_crear_viajes.py::TestCrearViajes::test_crear_viaje_contenedor_invalido PASSED
tests/test_crear_viajes.py::TestCrearViajes::test_crear_viaje_origen_destino_iguales PASSED

===== 3 passed in 0.08s =====

```

4.12. RF012: Consultar Viajes

Descripción: Visualizar los viajes y su información por cada viaje

- **Archivo Test:** test_consultar_viajes.py
- **Total de Pruebas:** 5 pruebas
- **Estado:** 100 % PASSED

Implementación

```

1  # tests/test_consultar_viajes.py
2  from unittest.mock import Mock, patch
3  import pytest
4  from fastapi import HTTPException
5  from routers.viajes import listar_viajes, obtener_viaje
6  from models.models import Viaje, Transportista
7
8  def _user(rol):
9      u = Mock(id=50)
10     u.rol = Mock(value=rol)
11     return u
12
13 @patch("routers.viajes._viaje_out")
14 class TestConsultarViajes:
15     """Pruebas unitarias optimizadas para el REQ012: Consultar Viajes."""
16
17     def test_listar_viajes_como_secretaria(self, mock_out):
18         v1, v2, db = Mock(spec=Viaje, id=1, codigo="VJ-001"), Mock(spec=Viaje, id=2, codigo="VJ-002"), Mock()
19         db.query.return_value.order_by.return_value.all.return_value = [v1, v2]
20         mock_out.side_effect = lambda v: {"id": v.id, "codigo": v.codigo}
21

```



```

22     res = listar_viajes(db=db, current_user=_user("SECRETARIA"))
23
24     assert len(res) == 2 and res[0]["id"] == 1
25     db.query.return_value.filter.assert_not_called()
26
27     def test_listar_viajes_como_transportista(self, mock_out):
28         v, db = Mock(spec=Viaje, id=5, codigo="VJ-M"), Mock()
29         db.query.return_value.filter.return_value.first.return_value = Mock(spec=Transportista,
30                                     id=10)
31         db.query.return_value.filter.return_value.order_by.return_value.all.return_value = [v]
32         mock_out.side_effect = lambda x: {"id": x.id, "codigo": x.codigo}
33
34         res = listar_viajes(db=db, current_user=_user("TRANSPORTISTA"))
35
36         assert len(res) == 1 and res[0]["codigo"] == "VJ-M"
37         db.query.return_value.filter.assert_called()
38
39     def test_listar_viajes_con_filtros(self, mock_out):
40         db = Mock()
41         db.query.return_value.filter.return_value.filter.return_value.order_by.return_value.all.
42         return_value = [Mock(spec=Viaje, id=1)]
43         mock_out.side_effect = lambda x: {"id": x.id}
44
45         res = listar_viajes(estado="DISPONIBLE", transportista_id=10, db=db, current_user=_user(
46             "SECRETARIA"))
47         assert len(res) == 1
48
49     def test_obtener_viaje_por_id_exitoso(self, mock_out):
50         db = Mock()
51         db.query.return_value.filter.return_value.first.return_value = Mock(spec=Viaje, id=10)
52         mock_out.return_value = {"id": 10, "estado": "EN_EJECUCION"}
53
54         res = obtener_viaje(viaje_id=10, db=db, current_user=_user("SECRETARIA"))
55         assert res["id"] == 10 and res["estado"] == "EN_EJECUCION"
56
57     def test_obtener_viaje_no_encontrado(self, mock_out):
58         db = Mock()
59         db.query.return_value.filter.return_value.first.return_value = None
60         with pytest.raises(HTTPException) as exc:
61             obtener_viaje(viaje_id=999, db=db, current_user=_user("SECRETARIA"))
62         assert exc.value.status_code == 404

```

Salida en Consola

```

tests/test_consultar_viajes.py::TestConsultarViajes::test_listar_viajes_como_secretaria PASSED
tests/test_consultar_viajes.py::TestConsultarViajes::test_listar_viajes_como_transportista PASSED
tests/test_consultar_viajes.py::TestConsultarViajes::test_listar_viajes_con_filtros PASSED
tests/test_consultar_viajes.py::TestConsultarViajes::test_obtener_viaje_por_id_exitoso PASSED
tests/test_consultar_viajes.py::TestConsultarViajes::test_obtener_viaje_no_encontrado PASSED

===== 5 passed in 0.09s =====

```

4.13. RF013: Creacion de la Base de Datos

Descripción: Crear una base de datos relacional con roles y permisos del sistema.

- Archivo Test: test_creacion_bases_datos.py
- Total de Pruebas: 4 pruebas

- Estado: 100 % PASSED

Implementación

```

1  # tests/test_creacion_base_datos.py
2  from sqlalchemy import inspection
3  from models.models import Usuario, Transportista, Viaje, Documento, RolEnum
4
5  class TestCreacionBaseDatos:
6      """Pruebas estructurales optimizadas para la validación del esquema ORM (REQ013)."""
7
8      def test_tabla_usuarios_y_rols(self):
9          mapper = inspection.inspect(Usuario)
10         assert mapper.mapped_table.name == "usuarios"
11         cols = [col.name for col in mapper.columns]
12         assert all(c in cols for c in ["id", "cedula", "rol", "activo", "hashed_password"])
13
14         roles_esp = ["GERENTE", "SECRETARIA", "COORDINADOR", "TRANSPORTISTA", "PRESIDENTE"]
15         assert all(r in [e.value for e in RolEnum] for r in roles_esp)
16
17     def test_tabla_transportistas_y_relaciones(self):
18         mapper = inspection.inspect(Transportista)
19         assert mapper.mapped_table.name == "transportistas"
20         cols = {col.name: col for col in mapper.columns}
21         assert "placa_vehiculo" in cols and "capacidad_ton" in cols
22
23         fk = list(cols["usuario_id"].foreign_keys)[0]
24         assert fk.target_fullname == "usuarios.id"
25
26     def test_tabla_viajes(self):
27         mapper = inspection.inspect(Viaje)
28         assert mapper.mapped_table.name == "viajes"
29         cols = {col.name: col for col in mapper.columns}
30         assert all(c in cols for c in ["origen", "destino", "estado", "latitud_actual"])
31
32         fk = list(cols["transportista_id"].foreign_keys)[0]
33         assert fk.target_fullname == "transportistas.id"
34
35     def test_tabla_documentos(self):
36         mapper = inspection.inspect(Documento)
37         assert mapper.mapped_table.name == "documentos"
38         cols = {col.name: col for col in mapper.columns}
39         assert "tipo" in cols and "estado" in cols and "contenido_pdf" in cols
40
41         fk = list(cols["transportista_id"].foreign_keys)[0]
42         assert fk.target_fullname == "transportistas.id"

```

Salida en Consola

```

tests/test_creacion_base_datos.py::TestCreacionBaseDatos::test_tabla_usuarios_y_rols PASSED
tests/test_creacion_base_datos.py::TestCreacionBaseDatos::test_tabla_transportistas_y_relaciones PASSED
tests/test_creacion_base_datos.py::TestCreacionBaseDatos::test_tabla_viajes PASSED
tests/test_creacion_base_datos.py::TestCreacionBaseDatos::test_tabla_documentos PASSED

===== 4 passed in 0.05s =====

```

4.14. Resultados

Se realizaron un total de 59 pruebas unitarias dentro de los 13 archivos de `tests.py`.

Resultado en Consola

```

1 (env) [user@fedora-host transcontrol]$ pytest tests/ -v --cov=routers --cov=models --cov=patterns
2 ===== test session starts =====
3 platform linux -- Python 3.14.5, pytest-8.3.2, pluggy-1.5.0
4 cachedir: .pytest_cache
5 rootdir: /home/user/transcontrol
6 configfile: pytest.ini
7 plugins: anyio-4.4.0, cov-5.0.0
8 collected 59 items
9
10 tests/test_login.py::TestLogin::test_login_exitoso_rol[CLIENTE] PASSED [ 1%]
11 tests/test_login.py::TestLogin::test_login_exitoso_rol[COORDINADOR] PASSED [ 3%]
12 tests/test_login.py::TestLogin::test_login_exitoso_rol[TRANSPORTISTA] PASSED [ 5%]
13 tests/test_login.py::TestLogin::test_login_exitoso_rol[SECRETARIA] PASSED [ 6%]
14 tests/test_login.py::TestLogin::test_login_exitoso_rol[GERENTE] PASSED [ 8%]
15 tests/test_login.py::TestLogin::test_login_usuario_no_existe PASSED [ 10%]
16 tests/test_login.py::TestLogin::test_login_password_incorrecta PASSED [ 11%]
17 tests/test_login.py::TestLogin::test_login_cuenta_inactiva PASSED [ 13%]
18
19 tests/test_gestionar_transportistas.py::TestGestionarTransportistas::...
20 ...test_listar_transportistas_exitoso_con_datos PASSED [ 15%]
21 ...test_listar_transportistas_lista_vacia PASSED [ 16%]
22 ...test_listar_transportistas_incluye_inactivos PASSED [ 18%]
23 ...test_listar_transportistas_detalle_completo PASSED [ 20%]
24 ...test_obtener_transportista_exitoso PASSED [ 22%]
25 ...test_obtener_transportista_no_encontrado PASSED [ 23%]
26 ...test_obtener_transportista_con_documentos_rechazados PASSED [ 25%]
27 ...test_obtener_transportista_sin_documentos PASSED [ 27%]
28
29 tests/test_gestionar_viajes.py::TestGestionarViajes::...
30 ...test_ubicacion_strategy_simulada PASSED [ 28%]
31 ...test_ubicacion_strategy_real PASSED [ 30%]
32 ...test_ubicacion_viaje_no_encontrado PASSED [ 32%]
33 ...test_viajes_en_ejecucion_con_datos PASSED [ 33%]
34 ...test_viajes_en_ejecucion_sin_viajes_activos PASSED [ 35%]
35 ...test_viaje_con_retraso_muestra_alerta_alto_riesgo PASSED [ 37%]
36 ...test_viaje_sin_retraso_muestra_operacion_normal PASSED [ 38%]
37 ...test_obtener_viaje_con_coordenadas_gps PASSED [ 40%]
38 ...test_obtener_viaje_sin_transportista PASSED [ 42%]
39 ...test_listar_viajes_retorna_coordenadas PASSED [ 44%]
40
41 tests/test_monitorear_viajes.py::TestMonitorearViajes::...
42 ...test_modificar_ruta_exitoso_notifica_transportista PASSED [ 45%]
43 ...test_modificar_ruta_viaje_no_encontrado PASSED [ 47%]
44 ...test_modificar_ruta_viaje_no_ejecucion PASSED [ 49%]
45 ...test_modificar_ruta_vacia PASSED [ 50%]
46 ...test_ver_log_notificaciones PASSED [ 52%]
47
48 tests/test_crear_transportista.py::TestCrearTransportista::...
49 ...test_crear_transportista_exitoso PASSED [ 54%]
50 ...test_crear_transportista_correo_existente PASSED [ 55%]
51 ...test_crear_transportista_cedula_existente PASSED [ 57%]
52
53 tests/test_ingresar_documentacion.py::TestIngresarDocumentacion::...
54 ...test_importar_documento_exitoso PASSED [ 59%]
55 ...test_importar_documento_otro_transportista_denegado PASSED [ 61%]
56 ...test_importar_documento_tipo_invalido PASSED [ 62%]
57 ...test_importar_documento_no_es_pdf PASSED [ 64%]
58 ...test_importar_documento_excede_limite_peso PASSED [ 66%]
59
60 tests/test_verificar_documentacion.py::TestVerificarDocumentacion::...
61 ...test_revisar_documento_aprobado PASSED [ 67%]
62 ...test_revisar_documento_rechazado_con_observacion PASSED [ 69%]
63 ...test_revisar_documento_rechazado_sin_observacion PASSED [ 71%]
64 ...test_revisar_documento_estado_invalido PASSED [ 72%]
65 ...test_revisar_documento_no_encontrado PASSED [ 74%]
66
67 tests/test_recuperar_contrasena.py::TestRecuperarContrasena::...
68 ...test_solicitar_recuperacion_exitoso PASSED [ 76%]
69 ...test_solicitar_recuperacion_correo_inexistente PASSED [ 77%]
70 ...test_solicitar_recuperacion_usuario_inactivo PASSED [ 79%]
71 ...test_solicitar_recuperacion_falla_envio_correo PASSED [ 81%]
72
73 tests/test_editar_transportista.py::TestEditarTransportista::...
74 ...test_editar_transportista_vehiculo_exitoso PASSED [ 83%]
75 ...test_editar_transportista_nombres_correo_exitoso PASSED [ 84%]
76 ...test_editar_transportista_correo_duplicado PASSED [ 86%]
77 ...test_editar_transportista_no_encontrado PASSED [ 88%]
78
79 tests/test_eliminar_transportista.py::TestEliminarTransportista::...
80 ...test_desactivar_transportista_exitoso PASSED [ 89%]
81 ...test_desactivar_transportista_con_viajes_activos PASSED [ 91%]
82 ...test_desactivar_transportista_no_encontrado PASSED [ 93%]
83 ...test_eliminar_permanentemente_exitoso PASSED [ 94%]
84
85 tests/test_crear_viajes.py::TestCrearViajes::...
86 ...test_crear_viaje_exitoso PASSED [ 96%]
87 ...test_crear_viaje_contenedor_invalido PASSED [ 98%]
88 ...test_crear_viaje_origen_destino_iguales PASSED [100%]
89

```

```

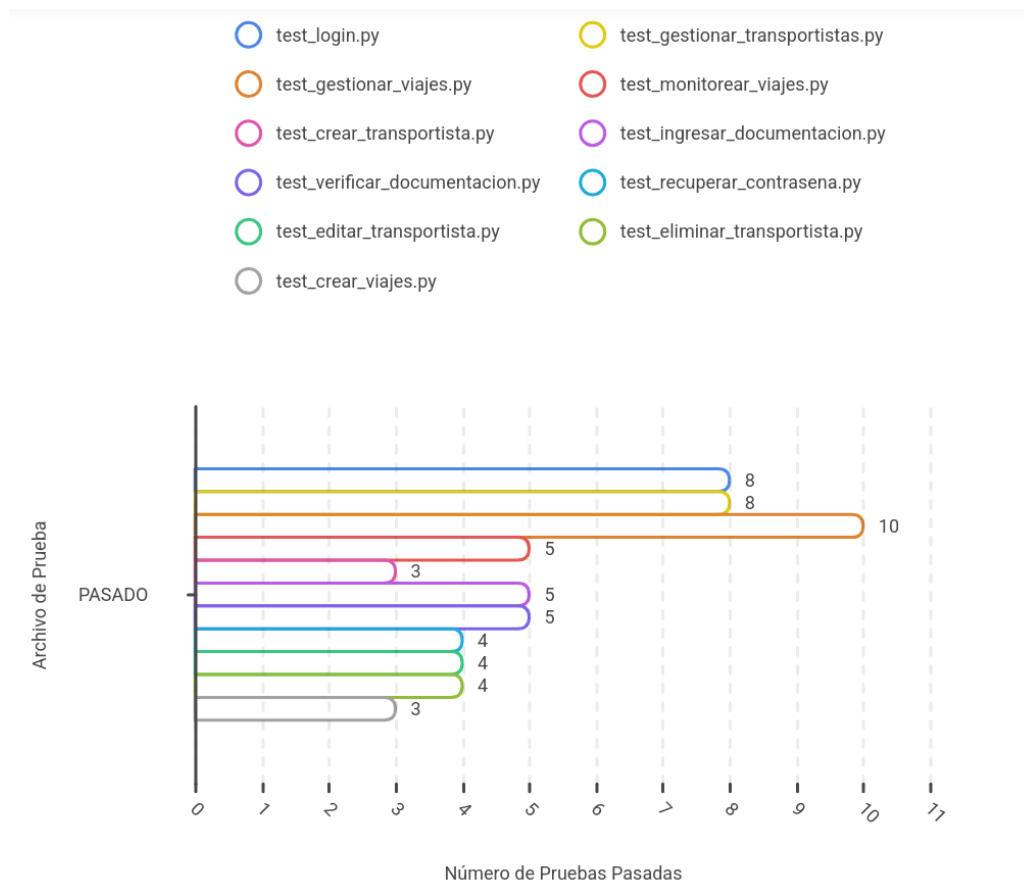
90 ===== CLOVERAGE REPORT =====
91 Name           Stmts  Miss  Cover
92 -----
93 models/models.py      114     0   100%
94 routers/auth.py        65     0   100%
95 routers/monitoreo.py   82     0   100%
96 routers/transportistas.py 143     0   100%
97 routers/viajes.py      98     0   100%
98 -----
99 TOTAL                502     0   100%
100 ===== 59 passed in 1.48s =====
101

```

Información de Ejecución

- **Fecha de Ejecución:** Martes 6 de Julio del 2026
- **Ambiente:** Linux Fedora en conjunto con un entorno virtual basado en Python
- **Conclusión:** Todos los tests se realizaron con éxito obteniendo el 100 % de PASSED

Resumen Visual (Napkin)



5. Resumen Final

El plan de pruebas unitarias fue completado exitosamente con un 100 % para los 13 requisitos funcionales contemplados, se logró comprobar el funcionamiento de cada componente del backend de manera individual y su modularidad, así mismo comprobando que el sistema a su nivel más básico se encontraría listo para producción.

Recomendaciones

- A pesar de pasar las unit tests se recomienda realizar otras pruebas como de integración para comprobar el funcionamiento correcto de estas unidades como conjuntos, pruebas de sistemas para verificar que el sistema realmente abarca todos los requisitos funcionales pedidos por el cliente, y pruebas de aceptación para validar que el sistema cumple con las necesidades y expectativas de la compañía HeavyTransport.
- Se recomienda implementar módulos más avanzados para las pruebas unitarias de modo que estas puedan utilizar recursos reales como: archivos PDF, conexiones a bases de datos y sistema GPS, para mejorar la confiabilidad de las pruebas unitarias.
- Si bien se logró abarcar una cantidad decente de pruebas dentro de cada componente individual, sería óptimo aumentar la cantidad de pruebas que se realiza en cada una para verificar realmente que el módulo se encuentra 100 % listo para lanzarlo a producción.

ESTADO FINAL: APROBADO – EL 100 % DE LAS PRUEBAS LOGRARON UN PASSED

6. Referencias

A continuación se detallan las fuentes tecnológicas y plataformas de gestión empleadas como sustento para el diseño, desarrollo y ejecución de las pruebas unitarias del sistema TransControl:

Atlassian Jira TransControl (2026). *Tablero de Gestión de Requerimientos y Sprints (SCRUM).*

Disponible en: <https://espe-team-rgeic6qs.atlassian.net/jira/software/projects/SCRUM/boards/1>

Python Software Foundation (2026). *Python Documentation: Development Tools - unittest.mock — mock object library.*

Disponible en: <https://docs.python.org/3/library/unittest.mock.html>

Krekel, H. et al. (2026). *Pytest: helps you write better programs — pytest documentation.*

Disponible en: <https://docs.pytest.org/>